UNIT-3

1. Explain the ER Model with neat diagrams.
2. Explain different types of attributes and relationships in the ER Model.
3. Explain Specialization, Generalization,Aggregation and Inheritance with examples.
4. Draw an ER diagram for a Library/Hospital/Banking System.
5. Explain the implementation of Primary Key, Foreign Key, UNIQUE, NOT NULL, and CHECK constraints in SQL with suitable examples.
6. What are integrity constraints? Explain Entity Integrity, Referential Integrity, and Domain Integrity constraints with suitable SQL examples.
7. Explain Nested and Correlated Subqueries with examples.
8. Explain different types of SQL Joins with suitable examples.
9. Explain SQL Views. Differentiate between updatable and non-updatable views.
10. Explain Relational Set Operations with suitable examples.

1. ER Model (Entity - Relationship Model)

1. Introduction :

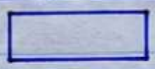
The Entity - Relationship (ER) model is a high level conceptual data model used to design a database. It helps us to represent the real world in the form of entities, their attributes and the relationships between them.

2. Components of ER Model :

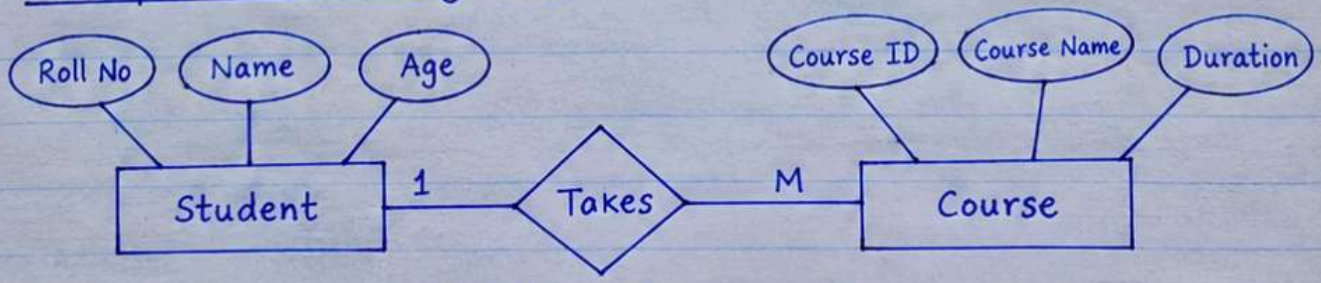
1) Entity - A real world object or thing (e.g., Student, Course).
2) Attribute - A property of an entity (e.g., Name, Age).
3) Relationship - Shows how entities are related to each other (e.g., Student takes Course).

3. Notations used in ER Model :

The ER model is represented using simple symbols.

Symbol	Meaning
	Entity
	Attribute
	Relationship
	Connection line

4. Example of ER Diagram :



(Student takes Course)

5. Conclusion :

The ER model is a useful tool for designing databases. It helps to represent entities, attributes and relationships in a simple and clear way.

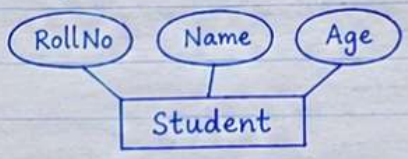
2. Explain different types of attributes and relationships in the ER Model. (4 Marks)

1. Types of Attributes :

Attributes are the properties or characteristics of an entity. In the ER model, attributes can be of different types.

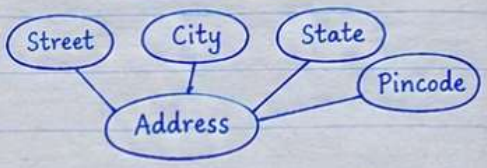
a) Simple (Atomic) Attribute :

- It cannot be divided into smaller parts.
- Example : Roll No, Name, Age.



b) Composite Attribute :

- It can be divided into smaller parts.
- Example : Address (Street, City, State, Pincode).



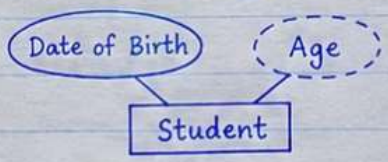
c) Multivalued Attribute :

- It can have more than one value for an entity.
- Example : Phone No, Skills.



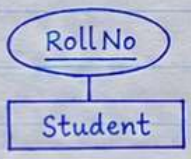
d) Derived Attribute :

- It is calculated from other attribute(s).
- Example : Age (derived from Date of Birth).



e) Key Attribute :

- It uniquely identifies an entity in an entity set.
- Example : Roll No, Employee ID.



2. Types of Relationships :

A relationship shows how entities are related to each other. The types of relationships in the ER model are :

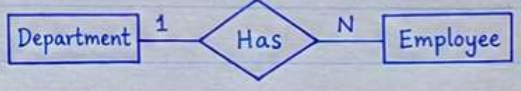
a) One-to-One (1:1) Relationship :

- One record in A is related to only one record in B, and vice versa.
- Example : One-to-one relationship between "Person" and "Passport".



b) One-to-Many (1:N) Relationship :

- One record in A can be related to many records in B, but each record in B is related to only one record in A.
- Example : One "Department" has many "Employees".



c) Many-to-Many (M:N) Relationship :

- Many records in A can be related to many records in B,
- Example : Student and Course (a student can take many courses and a course can have many students).



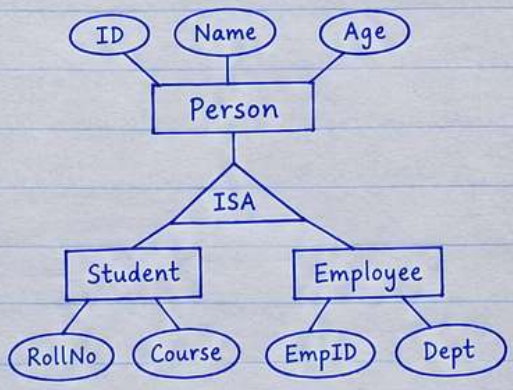
Conclusion :

Attributes describe the properties of entities and relationships show the connection between entities. Different types of attributes and relationships help us to design a clear, organized and meaningful database in the ER model.

3. Explain Specialization, Generalization, Aggregation and Inheritance with examples. (4 Marks)

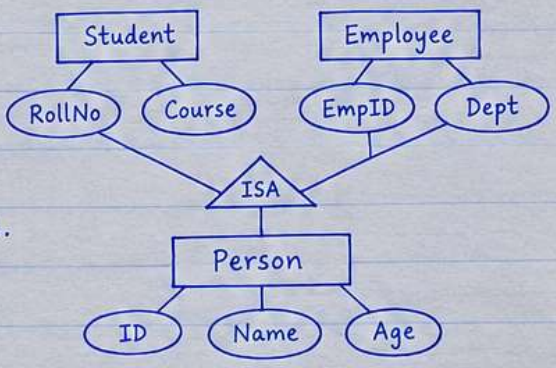
1. Specialization :

- It is a top-down process.
- It divides a general entity into specific sub-entities based on some conditions or properties.
- Example : Person → Student, Employee.



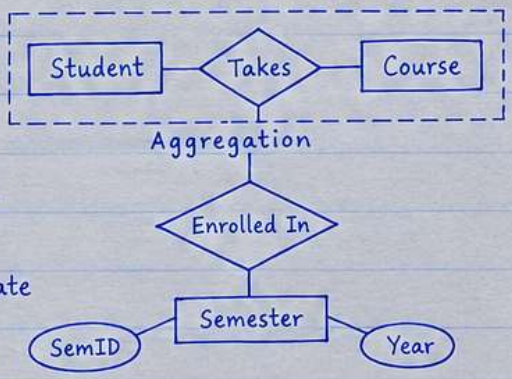
2. Generalization :

- It is a bottom-up process.
- It combines specific entities into a general entity.
- Example : Student + Employee → Person.



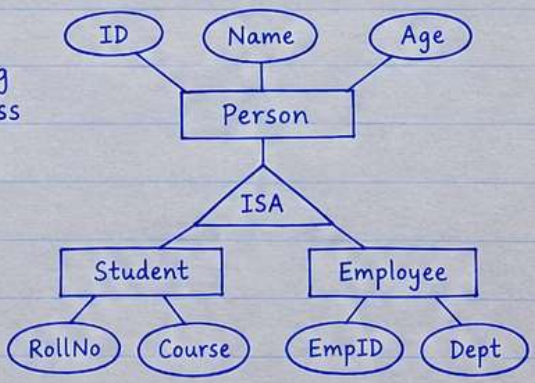
3. Aggregation :

- It is used to treat a relationship set as a higher-level entity.
- It is useful when a relationship has its own attributes or when we need to refer to the relationship as a single unit.
- Example : Student — Takes — Course
  (The relationship 'Takes' can participate in another relationship).



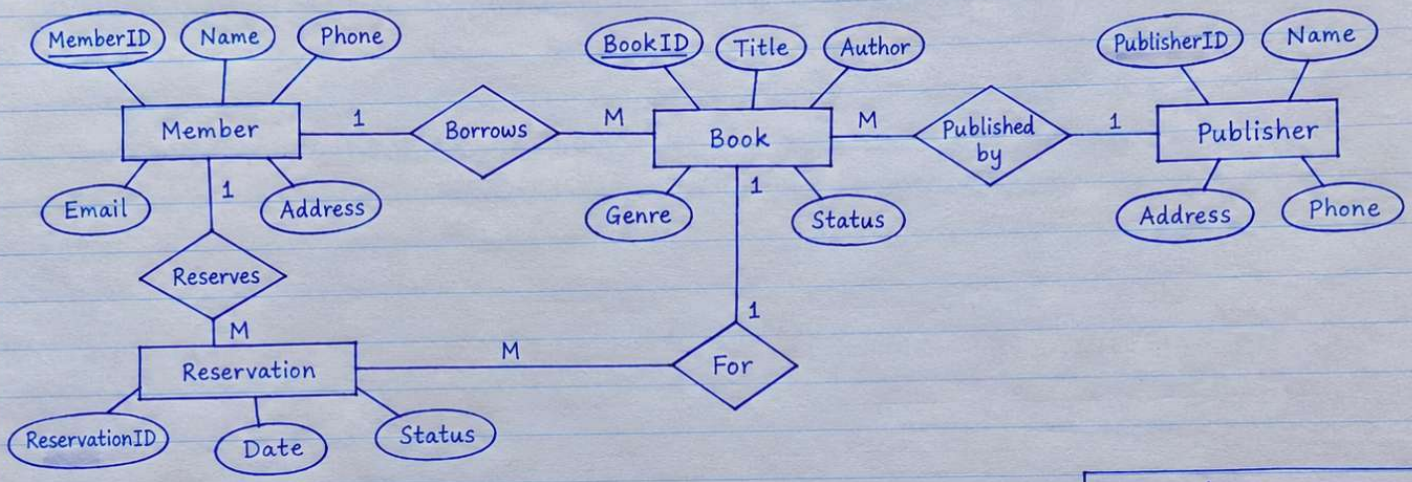
4. Inheritance :

- It is a concept from object-oriented design, but it is also used in ER modeling for extending the attributes of a superclass to its subclasses.
- A subclass inherits the common attributes of the superclass and can have its own attributes.
- Example : Student inherits ID, Name, Age from Person and adds RollNo, Course.



Conclusion : Thus, these concepts help represent complex entities and relationships clearly in an ER model.

ER Diagram for a Library System : (4 Marks)



Explanation :

1. **Member** – represents a library member. (MemberID, Name, Phone, Email, Address)
2. **Book** – represents a book in the library. (BookID, Title, Author, Genre, Status)
3. **Publisher** – represents the publisher of a book. (PublisherID, Name, Address, Phone)
4. **Reservation** – represents a book reservation by a member. (ReservationID, Date, Status)

Relationships :

1. **Member borrows Book** : One member can borrow many books, but each book is borrowed by many members (1 : M).
2. **Member reserves Book** : One member can reserve many books, but each book can be reserved by many members (1 : M).
3. **Book published by Publisher** : Many books can be published by one publisher, but each book is published by one publisher (M : 1).

Legend :

- ▭ = Entity
- ⬭ = Attribute
- ◇ = Relationship
- 1 = One (cardinality)
- M = Many (cardinality)

3. Explain the implementation of Primary Key, Foreign Key, UNIQUE, NOT NULL and CHECK constraints in SQL with suitable examples. (4 Marks)

1. Primary Key :

- A primary key uniquely identifies each record in a table.
- It does not allow duplicate values and cannot be NULL.
- A table can have only one primary key (single or composite).

Example :

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT
);
```

Students

StudentID	Name	Age
101	Ravi	20
102	Sita	21

2. Foreign Key :

- A foreign key creates a relationship between two tables.
- It refers to the primary key of another table.
- It does not allow values that are not present in the referenced table.

Example :

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    StudentID INT,
    OrderDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

Orders

OrderID	StudentID	OrderDate
1	101	2025-06-01
2	102	2025-06-02

3. UNIQUE :

- A UNIQUE constraint ensures that all values in a column are different.
- It allows NULL values (unlike primary key).

Example :

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE,
    Name VARCHAR(50)
);
```

Employees

EmpID	Email	Name
1	ravi@gmail.com	Ravi
2	sita@gmail.com	Sita

4. NOT NULL :

- A NOT NULL constraint does not allow a column to have a NULL value.
- It ensures that a value must be given when inserting a record.

Example :

```
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Price DECIMAL(10,2) NOT NULL
);
```

Products

ProductID	Name	Price
1	Laptop	50000
2	Mouse	1200

5. CHECK :

- A CHECK constraint ensures that the value in a column meets a given condition.
- It helps to keep data valid and correct.

Example :

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT CHECK (Age >= 18)
);
```

Employees

EmpID	Name	Age
1	Ravi	22
2	Sita	25

Conclusion :

These constraints help to maintain data integrity, avoid invalid data and keep the database consistent and reliable.

4. What are integrity constraints? Explain Entity Integrity, Referential Integrity and Domain Integrity constraints with suitable SQL examples. (4 Marks)

Introduction :

- Integrity constraints are rules that make sure the data in a database is correct, consistent and reliable.
- They prevent invalid data from being entered or stored in the database.
- There are three main types of integrity constraints :
  1) Entity Integrity 2) Referential Integrity 3) Domain Integrity

1. Entity Integrity :

- Entity integrity ensures that every table has a unique and non-null value in its primary key.
- It does not allow duplicate or null values in the primary key column.
- Example :

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT
);
```

StudentID	Name	Age
101	Ravi	20
102	Sita	21

(StudentID is the primary key, so it is unique and cannot be null.)

2. Referential Integrity :

- Referential integrity ensures that a foreign key value in one table must match the primary key value in another table.
- It maintains the relationship between the two tables.
- It does not allow an orphan record (a record without a matching value.).

Example :

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    StudentID INT,
    OrderDate DATE,
  FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
);
```

Student

StudentID	Name
101	Ravi
102	Sita

Orders

OrderID	StudentID
1	101
2	102

(StudentID in Orders must exist in Student table.)

3. Domain Integrity :

- Domain integrity ensures that the values in a column are within the allowed range or type (domain).
- It uses data types, CHECK constraints or other rules to restrict values.

Example :

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT CHECK (Age BETWEEN 18 AND 60)
);
```

Employee

EmpID	Name	Age
1	Ravi	22
2	Sita	25

(Age must be between 18 and 60.)

Conclusion :

- These integrity constraints help to maintain the quality, consistency and security of data in a database.

5. Explain Nested and Correlated Subqueries with examples. (4 Marks)

1. Nested Subqueries :

- A nested subquery is a subquery inside another query.
- It is used when we need to get a result from one query and use it in another query.
- The inner query is executed first, and then the outer query uses its result.

Example :

Find the names of students who have scored more than 80 marks in Maths.

```
SELECT Name
FROM Students
WHERE Marks > (SELECT AVG(Marks)
               FROM Students
               WHERE Subject = 'Maths');
```

Students

StudentID	Name	Marks	Subject
1	Ravi	90	Maths
2	Sita	70	Maths
3	Hari	85	Science
4	Anu	60	Maths

(First the inner query finds the average marks in Maths, then the outer query shows students who scored more than that.)

2. Correlated Subqueries :

- A correlated subquery is a subquery that depends on the outer query.
- It uses a value from the outer query in its WHERE clause.
- It is executed for each row of the outer query.

Example : Find the names of employees who work in the same department as 'Ravi'.

```
SELECT Name
FROM Employee e1
WHERE DepartmentID = (SELECT DepartmentID
                      FROM Employee e2
                      WHERE e2.Name = 'Ravi'
                      AND e2.DepartmentID = e1.DepartmentID);
```

Employee

EmpID	Name	DepartmentID
1	Ravi	10
2	Sita	10
3	Hari	20
4	Anu	30

(The inner query gets the department of 'Ravi'. For each row in the outer query, it checks if the employee is in the same department.)

Conclusion :

- Nested subqueries are independent and run first.
- Correlated subqueries depend on the outer query and run for each row.

Both are useful in SQL and help us get more accurate results from the database.

6. Explain different types of SQL Joins with suitable examples. (4 Marks)

Introduction:

- SQL joins are used to combine rows from two or more tables based on a common column between them.
- They help us to get data from multiple tables in a single result.
- There are different types of SQL joins. The main types are: INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS JOIN.

1. INNER JOIN:

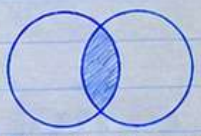
- Returns only the matching rows from both tables.
- If there is no match, the row is not included.

Example:

```
SELECT  Students.Name, Courses.CourseName
FROM    Students
INER JOIN Courses ON Students.CourseID = Courses.CourseID;
```

Result:

StudentID	Name	CourseID	CourseName
1	Ravi	101	Maths
2	Sita	102	Science
3	Hari	103	English



(Only matching data)

2. LEFT OUTER JOIN:

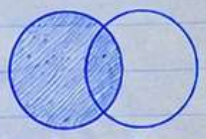
- Returns all rows from the left table and matching rows from the right table.
- If there is no match in the right table, NULL is shown.

Example:

```
SELECT  Students.Name, Courses.CourseName
FROM    Students
LEFT JOIN Courses ON Students.CourseID = Courses.CourseID;
```

Result:

StudentID	Name	CourseID	CourseName
1	Ravi	101	Maths
2	Sita	102	Science
3	Hari	103	English



(All from left table)

3. RIGHT OUTER JOIN:

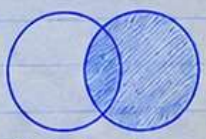
- Returns all rows from the right table and matching rows from the left table.
- If there is no match in the left table, NULL is shown.

Example:

```
SELECT  Students.Name, Courses.CourseName
FROM    Students
RIGHT JOIN Courses ON Students.CourseID = Courses.CourseID;
```

Result:

StudentID	Name	CourseID	CourseName
1	Ravi	101	Maths
2	Sita	102	Science
3	Hari	103	English



(All from right table)

4. FULL OUTER JOIN:

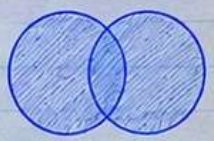
- Returns all rows when there is a match in either table.
- If there is no match, NULL is shown for missing values.

Example:

```
SELECT  Students.Name, Courses.CourseName
FROM    Students
FULL OUTER JOIN Courses ON Students.CourseID = Courses.CourseID;
```

Result:

StudentID	Name	CourseID	CourseName
1	Ravi	101	Maths
2	Sita	102	Science
3	Hari	103	English



(All rows from both tables)

5. CROSS JOIN:

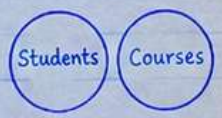
- Returns the Cartesian product (all possible combinations) of rows from both tables.
- It does not use any condition.

Example:

```
SELECT  Students.Name, Courses.CourseName
FROM    Students CROSS JOIN Courses;
```

Result:

StudentID	Name	CourseID	CourseName
1	Ravi	101	Maths
2	Sita	102	Science
3	Hari	103	English



(All combinations)

Conclusion:

- Different types of SQL joins help us to get the required data from multiple tables.
- We can choose the correct join based on our need.

7. Explain SQL Views. Differentiate between updatable and non-updatable views. (4 Marks)

1. SQL Views :

- A view is a virtual table in SQL.
- It is created from a SELECT query and does not store data itself.
- It shows data from one or more tables in a simplified form.
- We can use a view like a normal table (to select data).
- Views help in security, simplification and reusability of queries.

Example :

Create a view to show the names and marks of students.

```
CREATE VIEW StudentView AS
SELECT Name, Marks
FROM Students;
```

Now we can use th view like a table :

```
SELECT *
FROM StudentView;
```

2. Updatable Views :

- An updatable view is a view through which we can insert, update or delete data (if it follows certain rules).
- It is usually created from a single table (no joins, no group functions).
- The view must have a unique key (primary key) to identify the row.

Example : Create an updatable view :

```
CREATE VIEW EmpView AS
  SELECT EmpID, Name, Salary
  FROM Employees;
```

We can update data through this view :

```
UPDATE EmpView
SET Salary = 50000
WHERE EmpID = 101;
```

3. Non-Updatable Views :

- A non-updatable view cannot be used to insert, update or delete data.
- It is created using joins, group functions, DISTINCT, UNION, etc.
- These views are only for reading data.

Example : Create a non-updatable view (using join):

```
CREATE VIEW DeptEmpView AS
  SELECT e.Name, d.DeptName
  FROM Employees e, Departments d
  WHERE e.DeptID = d.DeptID;
```

Trying to update will give an error :

```
UPDATE DeptEmpView
SET DeptName = 'IT'
WHERE Name = 'Ravi';
```

(This will not work)

4. Difference between Updatable and Non-Updatable Views :

Feature	Updatable View	Non-Updatable View
1. Data modification	Can be used to insert, update, delete data.	Cannot be used to modify data.
2. Source table	Usually based on a single table.	Based on joins, group functions, DISTINCT, UNION, etc.
3. Primary key required	Must have a unique key.	Not required.
4. Purpose	Used for simple data manipulation.	Used for complex queries and data presentation only.

Conclusion :

- SQL views help us to access data in a simple way. Updatable views can be used to change data, while non-updatable views are only for viewing data.

Q. Explain Relational Set Operations with suitable examples. (4 Marks)

Ans: **Relational Set Operations :**

- Relational set operations are used to perform operations on two relations and produce a new relation as result.
- The basic set operations are UNION, INTERSECTION, DIFFERENCE, CARTESIAN PRODUCT and COMPLEMENT.

1. UNION (R ∪ S) :

- The union of two relations contains all the tuples that are in either relation R or relation S (or both).

Example :

R (Employee)

Eid	Name	Dept
101	Ravi	CSE
102	Sita	ECE

S (Employee)

Eid	Name	Dept
103	Hari	ME
102	Sita	ECE

R ∪ S :

Eid	Name	Dept
101	Ravi	CSE
102	Sita	ECE
103	Hari	ME

2. INTERSECTION (R ∩ S) :

- The intersection of two relations contains only the tuples that are common in both relations.

Example :

R (Employee)

Eid	Name	Dept
101	Ravi	CSE
102	Sita	ECE

S (Employee)

Eid	Name	Dept
101	Ravi	CSE
103	Hari	ME

R ∩ S :

Eid	Name	Dept
101	Ravi	CSE

3. DIFFERENCE (R − S) :

- The difference of two relations contains the tuples that are in R but not in S.

Example :

R (Employee)

Eid	Name	Dept
101	Ravi	CSE
102	Sita	ECE

S (Employee)

Eid	Name	Dept
103	Hari	ME
102	Sita	ECE

R − S :

Eid	Name	Dept
101	Ravi	CSE

4. CARTESIAN PRODUCT (R × S) :

- The Cartesian product of two relations combines every tuple of R with every tuple of S.

Example :

R (Student)

Sid	Name
1	Ajay
2	Balu

S (Course)

Cid	Cname
C1	DBMS
C2	Python

R × S :

Sid	Name	Cid	Cname
1	Ajay	C1	DBMS
1	Ajay	C2	Python
2	Balu	C1	DBMS
2	Balu	C2	Python

5. COMPLEMENT (R^c) :

- The complement of a relation R contains all the tuples from the universal set that are not in R.

Example : Universal set $U = \{1,2,3,4,5\}$

$R = \{1,2,3\}$

$R^c = \{4,5\}$

